Group Assignment Algorithms

Sorting Algorithms and Experiment Planning

Joel Isak

April 22, 2026

Contents

1 Introduction

This assignment is a part of the course TAIK12 Algorithms. The assignment focuses on the implementation and analysis of some fundamental sorting algorithms, as well as planning and executing experiments to evaluate their performance.

The main goal of this assignment is to develop a deeper understanding of how different sorting algorithms operate in practice, and how their performance can relate to the theoretical time complexities. In order to achieve this, three sorting algorithms were chosen and implemented in the programming language C, and their behavior was studied through experiments.

Furthermore, the assignment aims to introduce different empirical evaluation techniques by measuring the number of basic operations performed by each algorithms depending on input conditions. The results are then analyzed and compared to the theoretical expectations, allowing for better understanding of algorithm efficiency.

2 Experimentation

This experiment was conducted inside Visual studio code by counting the number of basic operation for each of the algorithms. The counter increased by one each time a comparison was made, resulting in quite big numbers for the slower algorithms.

We decided to test our algorithms using 4 differently ordered inputs to see if it would vary the results. These were Ordered Input, reverse ordered input, completely random input, and ordered input with 4 percent being random. For ordered and reverse ordered inputs we ran the program only once since they are deterministic. But for random and almost random inputs, we ran the program 30 times and took the average of these to get a more realistic answer.

Table 1: Input sizes and operations count for an algorithm.

Size n	**Operations op**
256	x
512	x
1024	x
2048	x
4096	x
8192	x
16384	x
32768	x

3 Results and Analysis

3.1 Selection Sort

Selection sort is a brute force type of algorithm with the time complexity of

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \in \Theta(n^2) \tag{1}$$

This is because of the comparison inside the two loops that runs (n-2)(n-1) times making it approximately n^2. As seen in the graph, this means that the basic operation count for small sizes makes little impact, but grows very quickly as the input size increases.

Running the theoretical result of n^2 on the input size of 256 gives 65536 which is two times more then we got in our test results as seen in Table 2. This pattern continues for the other input sizes which makes the multiplicative constant 1/2. And the exact complexity $((n-1)n)/2$

Furthermore we can see in our test results and the Fig. 1 graph that the basic operation count does not change if we change the type of ordered input. Meaning the program will run through the entire loop n times no mater how the input is organized. This concludes that the algorithm's best, average, and worst case all are of $\theta(n^2)$

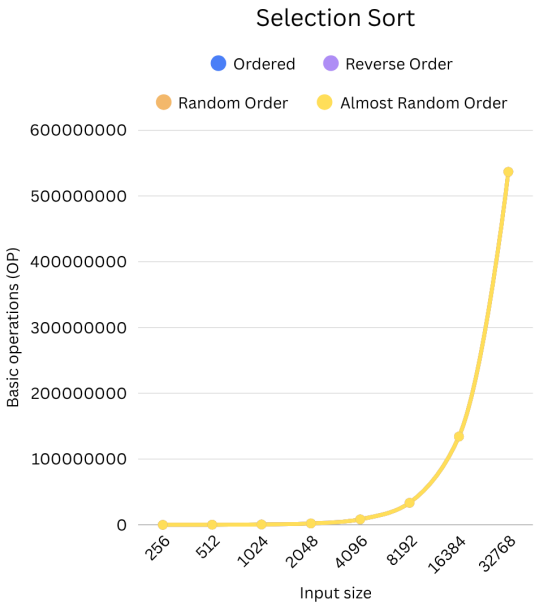

Figure 1: Selection Sort

Size	**Theoretical OP**	**Empirical OP**
256	65 536	32 385
512	262 144	130 305
1024	$1 * 10^6$	522 753
2048	$4 * 10^6$	$2 * 10^6$
4096	$16 * 10^6$	$8 * 10^6$
8192	$67 * 10^6$	$33 * 10^6$
16 384	$268 * 10^6$	$134 * 10^6$
32 768	$1 * 10^9$	$536 * 10^6$

Table 2: Selection Sort

3.2 Insertion Sort

In this subsection, we analyze the empirical performance of Insertion Sort for different types of input: ordered, reverse ordered, randomized and almost ordered input. The results are presented as lines on a graph where the number of basic operations is plotted against input size n.

For the ordered input, the number of operations grows linearly along with the input size. When n = 32768, the number of operations is 32767, which is n - 1. This shows that each element only requires one comparison, resulting in the time complexity θ(n). The multiplicative constant is approximately 1/2, since the number of operations follows T(n) = n(n-1)/2, this matches the theoretical worst-case complexity.

For input arranged in reverse order, the operation count grows quadratically. Doubling n causes the number of operations to increase by roughly a factor of four. This behavior matches T(n) = $(n^2)/2$, confirming a worst-case complexity of $\theta(n^2)$.

For randomized input, the behavior is also quadratic but with fewer operations than the worst case. The results are close to T(n) = $n^2/4$, which agrees with the average-case complexity $\theta(n^2)$.

For almost ordered input, the growth is between linear and quadratic. The number of operations is much lower than for random input, showing that Insertion Sort performs efficiently when almost all the data is sorted. Overall, the empirical results closely math the theoretical times we see in the textbook, which is linear time for best case and quadratic time for average and worst cases. The experiment also highlights the importance of the ordering of the input data.

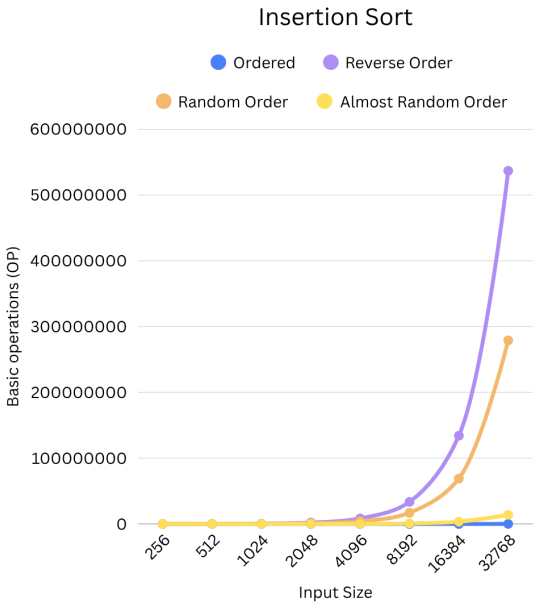

Figure 2: Insertion Sort

Size	Theoretical OP	Empirical OP
256	16 384	16 609
512	65 536	66 313
1024	262 144	260 611
2048	$1 * 10^6$	$1 * 10^6$
4096	$4 * 10^6$	$4 * 10^6$
8192	$16 * 10^6$	$16 * 10^6$
16 384	$67 * 10^6$	$67 * 10^6$
32 768	$268 * 10^6$	$268 * 10^6$

Table 3: Insertion Sort

3.3 Merge Sort

The divide and conquer algorithm Merge Sort ran many times faster than both selection sort and insertion sort for all types of input. Where given an array in random order of size 256 the algorithm needed only 1726 comparisons. Whereas selection sort needed 32385 comparisons for the same input, making merge sort about 19 times faster for this input size. That number only grows more and more the larger the size grows.

As seen in the graph below, the biggest operations count for merge sort was just

under $4.5 * 10^6$, where for selection sort the highest count was $1 * 10^9$. This is because selection sort grows exponentially while merge sort grows in $n * log(n)$ according to the theoretical time complexity explained by Anany Levitin.

Comparing this to the empirical results we got in our tests, we can see that the multiplicative constant is around 2,8-3,0 (*empirical*/*theoretical*).

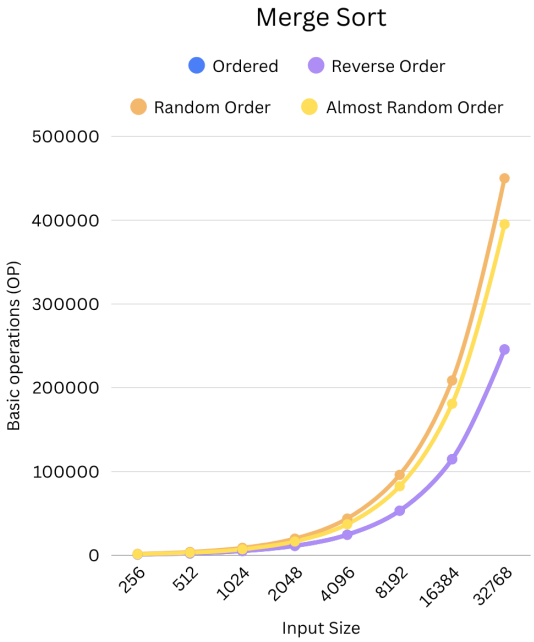

Figure 3: Merge Sort

Size	Theoretical OP	Empirical OP
256	617	1726
512	1387	3957
1024	3082	8943
2048	6781	19 940
4096	14 796	43 970
8192	32 058	96 136
16 384	69 049	208 680
32 768	147 962	450 102

Table 4: Merge Sort

Moreover we can see according to the graph that merge sort grows similarly no matter the order of input. This makes us draw the conclusion that merge sort is of the time complexity $\theta(n * log(n))$ for best, worst and average case

4 Conclusions

In conclusion, the experiments provided a clear comparison between the empirical results and the theoretical time complexities of the sorting algorithms. Overall, the results closely matched the expected behavior. Selection sort showed consistent quadratic growth regardless of input, confirming the $\theta(n^2)$ complexity in all cases. Insertion Sort varied depending on input, performing linearly for ordered data and quadratically for reverse and random inputs, demonstrating the sensitivity to its input order. Merge sort was the most efficient, with growth following $n * log(n)$ and small dependence on input type.

5 References

@Book levitin11,

author = Anany Levitin,

title = Introduction to the Design and Analysis of Algorithms,

edition = Third,

year = 2011,

publisher = Pearson Education